

Цель работы

Изучить ключевые архитектурные и функциональные особенности аналитической колоночной СУБД ClickHouse, непосредственно влияющие на производительность и логику работы с данными:

- колоночное хранение и его влияние на скорость аналитических запросов;
- разреженный первичный индекс (sparse primary index) и его отличие от плотных индексов в строковых СУБД;
- механизм асинхронных мутаций, заменяющий традиционные операции UPDATE/DELETE.

Научиться создавать таблицы семейства **MergeTree**, загружать и запрашивать значительные объёмы данных, анализировать планы и метрики выполнения запросов, а также осознанно применять мутации для модификации данных.

Задачи

1. Подготовка окружения и создание таблицы.

- Запустить ClickHouse (Docker, локально или облачная песочница).
- Подключиться через **clickhouse-client** или веб-интерфейс.
- Выполнить готовый DDL: создать таблицу **test_events** типа MergeTree с полями:
event_date Date, user_id UInt32, event_type LowCardinality(String), value Float32,
и первичным ключом **ORDER BY (event_date, user_id)**.
- Убедиться, что таблица создавалась командой **SHOW CREATE TABLE test_events**.

2. Загрузка небольшого, но наглядного объёма данных.

- Вставить 200–500 тысяч строк, используя готовый скрипт **INSERT INTO test_events SELECT ... FROM numbers(200000)** с генерацией случайных значений (функции **rand()**, **today() - rand() % 365** и пр.), который предоставляется в методичке.
- Проверить количество строк через **SELECT count()**.

3. Колоночное хранение: экономное чтение.

- Выполнить два запроса и замерить их время:
 - **SELECT * FROM test_events WHERE event_date = '2024-01-01'**
 - **SELECT event_date, sum(value) FROM test_events WHERE event_date = '2024-01-01' GROUP BY event_date**
- Для каждого запроса включить измерение (**clickhouse-client** опция **--time** или засечь вручную).
- Записать времена выполнения, убедиться, что второй запрос (с агрегацией по одной колонке) работает быстрее, и объяснить это тем, что ClickHouse читает только нужные файлы колонок, а не все столбцы.

4. Разреженный первичный индекс: фильтрация по ключу и без ключа.

- Выполнить два запроса и сравнить скорость:
 - Фильтрация по ключу: **SELECT count() FROM test_events WHERE event_date = '2024-01-01'**

- Фильтрация по неключевой колонке: `SELECT count() FROM test_events WHERE value > 50000`
- (Необязательно, если интерфейс позволяет) Посмотреть план запроса через `EXPLAIN` и увидеть разницу: по ключу читается несколько гранул, по `value` — все.
- В отчёте зафиксировать времена и вывод: первичный ключ содержит метки для групп строк (гранул), поэтому поиск по ключу гораздо эффективнее, хотя и не такой точечный, как в обычных БД.

5. Асинхронные мутации: изменение данных «поверх» старых.

- Выполнить команду мутации: `ALTER TABLE test_events UPDATE value = value * 1.1 WHERE event_type = 'purchase'` (предварительно убедиться, что такие строки есть).
- Сразу после этого выполнить `SELECT * FROM system.mutations WHERE table = 'test_events'` и увидеть строку мутации с `is_done = 0`.
- Подождать завершения (дождаться `is_done = 1`) и снова запросить изменённые строки, сравнить значения `value` до и после.
- Кратко пояснить, что ClickHouse не переписывает строки на лету, а создаёт новые части данных в фоне, поэтому мутация асинхронна и не блокирует чтение.

6. Сводка особенностей и мини-вывод.

- Письменно перечислить три изученные особенности (колоночность, разреженный индекс, асинхронные мутации) и для каждой написать одно предложение, как она влияет на использование ClickHouse.
- Предложить простую аналогию (например, индекс как оглавление книги, где каждая запись – глава, а не страница) для объяснения одной из особенностей будущим студентам.

Формат результатов и содержание отчёта

Отчёт оформляется в электронном виде (PDF/DOCX) и должен содержать следующие разделы:

1. Титульный лист

- Название дисциплины, тема лабораторной работы, ФИО студента, группа, дата.

2. Цель работы

- Переписанная своими словами цель из методических указаний.

3. Ход работы и результаты

По каждой из задач (1–6) необходимо представить:

Задача	Что предъявить в отчёте
1. Подготовка окружения и создание таблицы	Скриншот успешного подключения к ClickHouse; полный DDL созданной таблицы (<code>SHOW CREATE TABLE test_events</code>) с кратким пояснением типов данных и ключа.

Задача	Что предъявить в отчёте
2. Загрузка данных	Команда INSERT (или ссылка на использованный скрипт) и скриншот результата SELECT count() после загрузки.
3. Колоночное хранение	Текст двух запросов и два скриншота с замеренным временем выполнения (или вывод clickhouse-client с опцией --time). Краткий вывод: почему запрос с агрегацией по одной колонке быстрее, чем SELECT * , со ссылкой на колоночное чтение.
4. Разреженный первичный индекс	Текст запросов с фильтрацией по ключу и по неключевому полю; замер времени для каждого. По желанию — вывод EXPLAIN , показывающий количество гранул. Вывод о влиянии разреженного индекса на скорость фильтрации.
5. Асинхронные мутации	Команда ALTER TABLE ... UPDATE . Скриншот состояния system.mutations сразу после мутации (с is_done=0) и после завершения (is_done=1). Сравнение значений value до и после мутации. Пояснение, почему изменения не применились мгновенно, и как это связано с перезаписью партов.
6. Сводка особенностей	Три пункта с перечислением изученных особенностей и их влиянием (по одному предложению). Одна педагогическая аналогия (на выбор) для объяснения студентам колледжа.

4. Выводы

- Общий вывод о том, чем ClickHouse принципиально отличается от традиционных реляционных СУБД (хотя бы 2-3 предложения).

Дополнительные требования:

- Все скриншоты должны быть читаемыми, содержать временные метки или явные замеры времени.
- В тексте отчёта допускается минимальное форматирование, но обязательна логическая связность.

Оцениваемые результаты

Лабораторная работа считается **зачтённой**, если:

1. Выполнены все 6 задач и по каждой представлены требуемые артефакты.
2. Замеры времени в задачах 3 и 4 убедительно демонстрируют заявленный эффект (второй запрос быстрее первого, фильтрация по ключу быстрее фильтрации по неключевому полю).
3. Зафиксирован асинхронный характер мутации (наличие строки в **system.mutations** с **is_done=0** после отправки **ALTER**).
4. Приведена корректная аналогия для будущих учащихся, показывающая понимание одной из особенностей на концептуальном уровне.
5. Выводы не противоречат изученным фактам и не копируют дословно методичку.